Análise de Desempenho das Cláusulas `collapse` e `schedule`

Simulação da Equação de Difusão 2D

Werbert Arles de Souza Barradas

Universidade Federal do Rio Grande do Norte (UFRN)
Disciplina de Programação Paralela - DCA3703

28 de setembro de 2025

Agenda

1. Introdução e Estratégias
2. Implementações
3. Validação do Código Serial
4. Resultados e Análise
5. Conclusão

Contexto e Estratégias Avaliadas

Problema

O custo computacional de simulações científicas está concentrado em **laços aninhados**. O objetivo é analisar o impacto de diferentes estratégias de paralelização de laços em OpenMP para um simulador de difusão 2D, com 6000 repetições(NT=6000)e grade de 512x512(NX=NY).

Versões Analisadas

1. **Serial:** Implementação sequencial para baseline.
2. **Paralelismo Ingênuo:** Múltiplas regiões paralelas (alto overhead).
3. **Otimização Parcial:** Região paralela única, mas **sem** `collapse`.
4. **Otimização Completa:** Região única **com** `collapse(2)` e teste das políticas de `schedule`.

1. Implementação Serial (Baseline)

Propósito

Estabelecer o tempo de execução base (T_{serial}). Todos os laços são sequenciais.

```
// Loop de tempo PRINCIPAL - SERIAL
for (int t = 0; t < NT; t++) {
    // La os espaciais - SERIAIS
    for (int i = 1; i < NX-1; i++) {
        for (int j = 1; j < NY-1; j++) {
            // ... c lculo do Laplaciano ...
        }
    }
}
```

2. Paralelização Ingênua

Estratégia e Gargalo

Aplicar `#pragma omp parallel for` em cada laço de trabalho. **Gargalo:** Alto overhead de criar e destruir threads (fork/join) 3x a cada passo de tempo.

```
// Loop de tempo PRINCIPAL - SERIAL
for (int t = 0; t < NT; t++) {
    // GERA OVERHEAD DE FORK/JOIN AQUI
    #pragma omp parallel for
    for (int i = 1; i < NX-1; i++) { /*...*/ }

    // E AQUI...
    #pragma omp parallel for
    for (int i = 0; i < NX; i++) { /*...*/ }

    // E AQUI NOVAMENTE...
    #pragma omp parallel for
    for (int j = 0; j < NY; j++) { /*...*/ }
}
```

3. Otimização Parcial (sem collapse)

Estratégia e Gargalo

Criar uma **única região paralela** para eliminar o overhead de fork/join. **Gargalo Remanescente:** Sem `collapse`, o OpenMP só distribui as linhas (laço `i`), limitando o balanceamento de carga.

```
// Loop de tempo PRINCIPAL - PARALELO
#pragma omp parallel
{
    for (int t = 0; t < NT; t++) {
        // UMA  NICA  CRIA  O DE THREADS AQUI

            // AINDA SEM COLLAPSE: paraleliza apenas o la o 'i'
            #pragma omp for schedule(static)
            for (int i = 1; i < NX-1; i++) {
                for (int j = 1; j < NY-1; j++) { /*...*/ }
            }
            #pragma omp for schedule(static)
            for (int i = 0; i < NX; i++) { /*...*/ }
    }
} // FIM DA REGI O PARALELA
```

4. Paralelização Otimizada (com collapse + schedule)

Estratégia Final

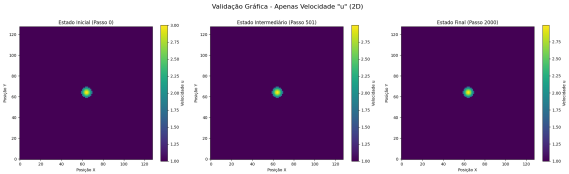
Manter a região paralela única e adicionar `collapse(2)` para tratar a grade 2D como um único bloco de trabalho, maximizando o potencial de balanceamento de carga.

```
// Loop de tempo PRINCIPAL - PARALELO
#pragma omp parallel
{
    for (int t = 0; t < NT; t++) {
            // MELHOR BALANCEAMENTO DE CARGA
            #pragma omp for collapse(2) schedule(static)
            for (int i = 1; i < NX-1; i++) {
                for (int j = 1; j < NY-1; j++) { /*...*/ }
            }

            #pragma omp for schedule(static)
            for (int i = 0; i < NX; i++) { /*...*/ }

    }
}// FIM DA REGI O PARALELA
```

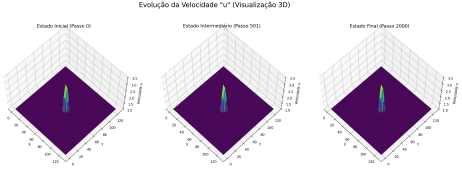
Validação Gráfica da Simulação

Análise de Coerência

Antes de analisar o desempenho, validamos se a simulação serial produz resultados fisicamente estáveis. Os gráficos mostram a evolução do campo de velocidade "u" em três instantes (passos 0, 501 e 2000), confirmando a estabilidade do código que servirá de *baseline*.



Visualização 2D



Visualização 3D

Resultado

A simulação se mostra estável, sem divergir ou apresentar artefatos, validando o código base.

Resultados Quantitativos (Tempos de Execução)

Estratégia	N. Threads	Tempo Médio (s)
Serial	**1**	**4.7540**
Paralelo Ingênua	16	3.0866
Otimizado (schedule + static)	16	2.8732
Otimizado (schedule + guided)	16	2.9334
Otimizado (collapse + static)	16	3.8546
Otimizado (collapse + guided)	16	3.7746
Otimizado (collapse + dynamic)	16	45.1201

Tabela: O melhor resultado (16 threads) é apresentado para as versões paralelas.

Comparativo de Desempenho (16 Threads)

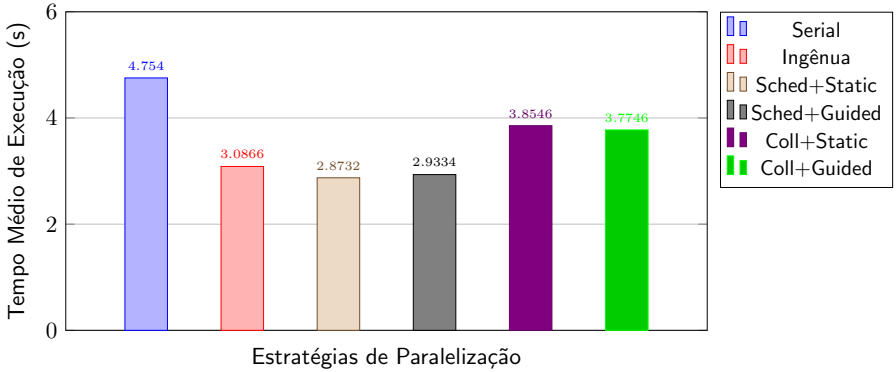


Figura: Comparativo do tempo de execução (16 threads). A estratégia `Sched+Static` apresentou o melhor desempenho.

Avaliação dos Resultados

Por que `static` foi rápido e `dynamic` foi lento?

A política de agendamento deve ser compatível com a natureza da carga de trabalho.

Cenário Ideal: Carga Uniforme

- **Problema:** Cada iteração do laço leva o mesmo tempo.
- **Solução:** `schedule(static)`.
- **Mecanismo:** O trabalho é dividido em blocos iguais **uma única vez**.
- **Resultado: Overhead mínimo**, melhor desempenho.

Cenário Inadequado: Carga Uniforme

- **Problema:** O mesmo, com carga uniforme.
- **Solução Incorreta:** `schedule(dynamic)`.
- **Mecanismo:** Cada thread pega **uma iteração de cada vez**.
- **Resultado: Overhead massivo**, desaceleração.

Conclusões

- **Estratégia > Micro-otimização:** A decisão de **quais laços** paralelizar foi o que gerou o maior ganho de desempenho.
- **O `schedule` é o fator mais crítico:** A comparação entre `static` (melhor desempenho) e `dynamic` (desaceleração acentuada) provou que a escolha do agendamento deve ser adequada à natureza da carga de trabalho.
- **Gargalos de Hardware:**A pequena diferença entre as versões otimizadas sugerem que o gargalo final não é o balanceamento de carga, mas sim a **largura de banda da memória** do sistema.